İŞLETİM SİSTEMLERİ

Mainboard (Ana Kart) Internal Components
+ CPU
+ DRAM
+ GPU
+ SSD, HDD (Storage)
+ Fan
+ Ethernet card (wifi, BU)
+ USB, HDMI, VGA, Sound Jack (IN/OUT)
+ PCI
+ BIOS*
+ ICs
+ PCU

- External Components
+ Display
+ Keyboards
+ Mouse
+ Speakers, mic
+ CAM
+ Joystick
+ Printer
+ External Storage (SSD, HDD...)
+ Pen driver
+ Scanners

1-) Donanımın Doğuşu: Transistör Devrimi:

- İlk transistör: 23 Aralık 1974'de Bell Laboratuvarlar'ında icat edildi.
- Mucitler: John Bardeen, William Shackley ve Walter Brattain.
- Bipolar Bağlantı Transistörü (BJT): Notlarda yaygın bir NPN transistörü olan 2N2222'nin Pin yapısı gösteriliyor.
- Gelişim Süreci: Transistör sayısı arttıkça işlem gücüde arttı.
  - Motorola 68040 (1990): 1,2 milyon transistör, 32-bit, 40MHz
  - Intel 8080 (1974): 4500 transistör, 8-bit, 2MHz
  - Intel 80486 (1989): 1,2 milyon transistör, 32-bit, 100MHz ve 16KB L1 önbellek

2-) İşletim Sistemi Mimarisi: Modlar ve Katmanlar:

- User Mode (Kullanıcı Modu): Web tarayıcısı, e-posta okuyucu ve müzik çalar gibi uygulamaların çalıştığı katmandır.
- Kernel Mode (Çekirdek modu): İşletim Sisteminin doğrudan donanımla konuştuğu, tam yetkili moddur.
- Sistem Hiyerarşisi:
  1-) Kullanıcı Arayüz Programı (User interface)
  2-) İşletim Sistemi (Operating system)
  3-) Donanım (Hardware)

3-) Öncü İşletim Sistemleri ve Yazılım Dilleri:

Gary Kildall ve CP/M:

- PL/M (1973): Mikroişlemciler için geliştirilen ilk yüksek seviyeli programlama dili ve derleyicisidir.
- CP/M (Control Program for Microcomputers): 1974'te Intel 8080 üzerinde disk sürücülerini kontrol edebilen ilk disk tabanlı işletim sistemidir.
- Önemi: BIOS kavramının öncüsüdür ve 5 yıl boyunca pazara hakim olmuştur.

Microsoft ve MS-DOS:

- 1980'lerde IBM PC'için geliştirilen DOS, Tim Peterson ve Microsoft (Bill Gates) liderliğinde yükseldi.
- MS-DOS: x86 Assembly diliyle yazılmış, komut satırı tabanlı (CLI) sistemidir. Notlarda dir, copy, format gibi temel komutlar ve disket yapıları görülüyor.

4-) Teknik Detay: (8080 Mimarisi):

- ALU (Aritmetik Mantık Birimi): Hesaplamaların yapıldığı merkez.
- Registers (Yazmaçlar): Accumulator (A), B, C, D, E, H, L gibi geçici veri depolama alanları.
- Program Counter: Çalıştırılacak bir sonraki komutun adresini tutar.
- Stack Pointer: Bellekteki yığın yapısını yönetir.

Bilgisayar Önyükleme (Booting the Computer):

- Bilgisayar baslatıldığında, BIOS baslatılır.
- İlk olarak sisteme ne kadar RAM yüklenmiş, klavye gibi basit çevre birimlerin kontrol eder ve yükler.
- PCIe ve PCI bağlantılarını kontrol eder ve takılı donanımları belirler. Yeni takılı cihazları yapılandırır.
- CMOS hafızada kayıtlı aygıtların listesini kontrol ederek BOOT (Önyükleme) aygıtını bulur. (HDD, SSD, CD, DISKET, USB-FLASH-MEMORY).
- Kullanıcı isterse BIOS yazılımında BOOT aygıtlarının sırasını ayarlayabilir.
- Önyükleme aygıtından ilk sektör belleğe (RAM) okunur ve yürütülür.
- BOOT aygıtının disk bölümlemesi analiz edilir ve hangi bölümün baslatılacağı belirlenir.
- İkinci ön yükleyici bu bölümden okunur ve çalıştırılır. Ardından isletim sistemin bu bölümden okunarak çalıştırılır.
- isletim sistemi daha sonra yapılandırma bilgilerini almak için BIOS'u sorgular. BIOS'un gördüğü her aygıtın, sürücü yazılımını yükleyerek, OS-kernel'e ekler. Böylece isletim sistemi ilgili aygıtlar ile konuşabilir.

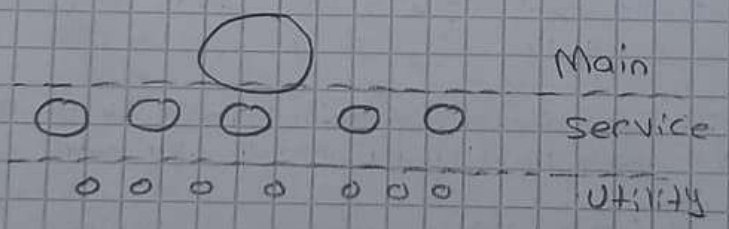
İşletim Sisteminin Yapısı:

1-) Monolitik Sistemler:

- isletim sistemi çekirdek modunda (kernel mode) tek bir program olarak çalışır.
- isletim sistemi, tek bir yürütülebilir ikili programda birbirine bağlanan bir prosedürler toplulugu olarak yazılır.
- Eğer ikincisi, öncekinin ihtiyaç duyduğu bazı yararlı hesaplamaları sağlıyorsa, sistemdeki her prosedür diğerini aramakta serbesttir.
- Herhangi bir prosedürü arayabilmek verimli görünsede birbirini çağırabilen binlerce prosedürün olması yavaş ve anlaşılması zor bir sisteme yol açabilir.
- Ayrıca, bu prosedürlerden birinin çökmesi tüm isletim sistemini çökertir.
- Bilgi gizleme açısından, esasen hiçbir prosedür yoktur her prosedür diğerleri tarafından görülebilir (bazıları hariç).
- Monolitik sistemlerde bile bazı yapılara sahip olmak mümkündür. OS tarafından sağlanan hizmetler, parametreler iyi tanımlanarak ardından bir tuzak komutu yürütülerek talep edilir. Bu komut makineyi kullanıcı'dan çekirdek moduna geçirir ve kontrol isletim sistemindedir.

+ İşletim Sisteminin Üç Temel Kısmı:

- istenen hizmet prosedürünü çağıran bir ana program.
- Sistem çağrılarını gerçekleştiren bir dizi servis prosedürü.
- Servis prosedürlerine yardımcı olan bir dizi yardımcı program prosedürü.



- Monolitik çekirdeğin yürütülmesi, bellek yönetimi, dosya yönetimi, vb. hizmetler aynı adres alanı altında uygulandığından oldukça hızlıdır.
- Bir işlem monolitik çekirdekteki tek adres alanında tamamen çalışır.
- Monolitik çekirdek, statik tek bir ikili dosyadır.
- " " herhangi bir hizmet başarısız olursa, tüm sistemin başarısız olmasına neden olur.
- Herhangi yeni bir hizmet eklenmek isterse tüm sistemin kullanıcı tarafından değiştirilmesi lazım.
- monolatik örnek: CP/M ve DOS

2-) Katmanlı (Layerd) Sistemler:

Layer (katman)	Function (iş)
5	Operatör
4	Kullanıcı Programı
3	Giriş/Çıkış Yönetimi
2	İletişim/Haberleşme
1	Bellek Yönetimi
0	İşlemci Tahsisi

* Katman 0, işlemcinin tahsisi ile ilgilenir, kesintiler yaşandığında veya zamanlayıcıların süresi dolduğunda süreçler arasında geçiş yapar.

* Katman 1, Memory yönetimi işini yönetir. İşlemcide yürütülen süreçler için ana hafızada yer ayırır.

* Katman 2, Her işlem ile operatör konsolu arasındaki iletişimi yönetir.

* Katman 3, I/O cihazlarını yönetme ve bunlara giden bilgi akışlarını ara belleğe alma ile ilgilenir.

* Katman 4, Kullanıcı programlarının bulunduğu yerdir.

|#| Katmanlar yerine MULTICS, istekilerin dıştakilerden daha ayrıcalıklı olduğu bir dizi eşmerkezli halkaya sahip olarak tanımlandı.

Microkernels (Mikro Çekirdekler):

Monolitik yapının aksine, her şeyi çekirdeğin içine tıkıştırmak yerine, çekirdeği olabildiğince küçük tutma mantığıdır.

-> Çekirdeğin içine sadece en kritik görevleri (işlemci yönetimi, bellek yönetimi...) koymak ve diğer herşeyi kullanıcı moduna taşımak.

-> örn: Ses sürücüsünde bir hata oluşursa sadece ses gider; sistemin tamamı çökmez çünkü sürücü çekirdek modunda değil, kullanıcı modunda sıradan bir program gibi çalışır.

-> Bir modül ele geçirilirse, tüm sisteme erişim sağlanamaz çünkü yetkileri kısıtlıdır.

İstemci - Sunucu Modeli: Mikroçekirdek yapısının bir üst aşamasıdır.

-> **Yapısı:** İşletim sistemleri hizmetleri birer "Sunucu" (server) gibi çalışır. Kullanıcı programları ise "İstemci" (client) olur.

-> **İletişim:** İstemci, bir işlem yapmak istediğinde çekirdek üzerinden sunucuya mesaj gönderir. Çekirdek sadece bu mesajlaşmaya aracılık eder.

-> **Esneklik:** Bu model, ağ üzerindeki farklı bilgisayarlar arasında da çalışabilir. Kullanıcı işlemin kendi bilgisayarında mı yoksa uzaktaki bir sunucuda mı yapıldığını farketmez.

Sanal Makineler (Virtual Machines): Donanımı tamamen taklit ederek tek bir fiziksel PC'de birden fazla işletim sistemi çalıştırma yöntemidir.

- **VMM (Virtual Machine Monitor):** Doğrudan donanım üzerinde çalışır ve üstteki işletim sistemlerine bir alan sunar.

-> Farklı işletim sistemlerini aynı anda denemek veya sunucu kaynaklarını verimli kullanmak.

Exokernels (Dış Çekirdekler): Geleneksel işletim sistemlerinden farklı olarak donanım kaynaklarını doğrudan kullanıcıya tahsis eder.

-> Çekirdek kaynakları yönetmek yerine sadece "Kimin hangi kaynağı kontrol etme hakkı var" onu kontrol eder.
-> Uygulamalar donanıma en yakın seviyede erişim sağladığı için çok hızlıdır.

İŞLEMLER VE Tredler (Processes and Threads):

-> Process (Süreç), yürütülmekte olan bir programdır.

-> Bir Processin bileşenleri şunlardır:
+ Yürütülecek olan program.
+ Program üzerinde çalışacağı veriler.
+ Programın ihtiyaç duyduğu kaynaklar
+ Yürütme durumu.
+ CPU kayıtçıları

Not: Bir Process, bir Programdan hem daha fazla hem de daha azdır.

Daha Fazladır (More): Bir Program, bir Processin bağlamının sadece bir parçasıdır.

Daha Azdır (Less): Bir Program, birden fazla Processi çağırabilir/başlatabilir.

Süreç Oluşturma (Process Creation):
- **Sistemin Başlatılması:** Bilgisayar ilk açıldığında işletim sistemi tarafından otomatik olarak süreçler oluşturulur.
- **Çalışan Bir Sürecin Başka Bir Süreç Oluşturması:** Yürütülmekte olan bir süreç, bir sistem çağrısı yaparak yeni bir süreç başlatabilir.
- **Kullanıcının Yeni Bir Süreç Başlatma İsteği:** Kullanıcının etkileşimli olarak yaptığı işlemlerle süreç oluşturulur.
- **Bir Batch (Yığın) İşleminin Başlatılması:** Arka planda sırayla çalışması planlanan işlerin (batch jobs) başlatılmasıyla süreçler meydana gelir.

Programlama: Uni-(tekli) ve Multi-(çoklu) Karşılaştırması:
-> Bazı sistemler aynı anda sadece tek bir sürecin yürütülmesine izin verir. Bunlara tekli programlama sistemleri denir.

-> Diğerleri birden fazla sürece, yani birçok sürecin eşzamanlı olarak yürütülmesine izin verir. Bunlara çoklu programlama sistemleri denir.

-> Bir çoklu programlama sisteminde, CPU her birini onlarca veya yüzlerce milisaniye çalıştırarak otomatik olarak süreçten sürece geçiş yapar. Gerçekte, CPU aynı anda sadece ve sadece bir süreci çalıştırmaktadır.

Yürütme Modeli (Execution model):

-> Yıllar içinde işletim sistemi tasarımcıları, eszamanlılıkla başa çıkmayı kolaylaştıran bir model geliştirdiler. Bu modelde, PC'deki çalıştırılabilir her yazılım bir dizi süreçe ayrılır ve her biri, yürütülecek bir sonraki talimatı gösteren bir işaretçiye sahip bir kod bloğu olarak görülür.

-> Birden fazla süreç tek bir CPU'yla nasıl paylaşılabilir

- Her sürecin çalışma şansı elde etmesi - adil zamanlama (fair scheduling)
- Birbirlerinin durumunu değiştirmemeleri - koruma (protection)

Süreç Durumları (Process States):

-> Bir süreçe atfedilebilecek bir dizi durum vardır: nitekim, bir çoklu programlama sisteminin işleyişi süreç durumları üzerindeki bir durum geçiş diyagramı ile açıklanabilir. Bir sürecin durumları şunlar. Kısaca:

- **Yeni (New):** oluşturulmakta olan ancak henüz çalıştırılabilir süreçler havuzuna dahil edilmemiş bir süreç.
- **Hazır (Ready):** fırsat verildiğinde çalıştırılmaya hazır olan süreçlerdir.
- **Aktif (Active):** şu anda CPU tarafından çalıştırılan süreç.
- **Engellendi (Blocked):** Belirli bir olay gerçekleşene kadar çalışmayan bir süreç.
- **Durdurulduu (Stopped):** sürecin operatör veya kullanıcı tarafından askıya alındığı, engellendi (blocked) durumunun özel bir hali.
- **Çıkış Yapıyor (Exiting):** Çalıştırılabilir süreçler havuzundan çıkarılmak üzere olan bir süreç.

Süreç Açıklaması (Process Description):

-> İşletim sistemi, süreçleri yönetmek ve kontrol etmek için onlar hakkında belirli bilgiler bilmek zorundadır. Bu tür bilgiler genellikle iki kategoriye ayrılır:

- Süreç durum bilgisi: örn; CPU kaydedicileri, Program sayacı
- Süreç kontrol bilgisi: örn; Zamanlama önceliği, elinde tutuğu kaynaklar...

* Bu süreç bilgileri koleksiyonu bir **Süreç Kontrol bloğunda (PCB)** tutulur ve buradan erişilir.

* Her iki gruptaki bilgilerde işletim sistemine bağımlıdır.

Süreç Zamanlama (Process scheduling):

-> Çoklu programlamanın amacı, her zaman çalışan bir kullanıcı sürecine sahip olmaktır. İşletim sistemi, aktif hale gelecek bir sonraki kullanıcı sürecini dinamik olarak seçerek CPU'yu üretken işlerle meşgul tutar.

-> Zamanlama işlemi, dağıtıcı (dispatcher) adı verilen modül tarafından gerçekleştirilir.

CPU'nun kontrolü (Control of the CPU):

-> CPU aynı anda sadece bir şey yapabilir. Bir kullanıcı süreci çalışırken dağıtıcı (dispatcher) çalışmaz, bu nedenle işletim sistemi kontrolü kaybeder.

* Dağıtıcı (Dispatcher) kontrolü nasıl geri kazanılır:
- İşlemi bittiğinde dağıtıcıyı uyandırması için sürece güvenmek
- Dağıtıcıyı uyandırmak için bir mekanizma sağlamak

-> İki yaklaşımdaki sorun, bazen süreçlerin sonsuz döngüye (loop) girmesidir. Bu nedenle, çalar saat kesmesi (alarm clock interrupt) yaklaşımı daha iyidir.

Bağlam Değişimi (Context Switch):

-> Bir olay gerçekleştiğinde, işletim sistemi aktif sürecin durumunu kaydeder ve Kesme servis rutinini (ISR-interrup service routine) durumunu geri yükler. Bu mekanizma Bağlam Değişimi (Context Switch) denir.

-> Neler kaydedilmelidir? Bir sonraki sürecin zarar verebileceği veya vereceği her şey

- Program sayıcı, Program Durum Sözcüğü (PSW), Dosya erişim işaretçileri, Bellek

* Durumu kaydederken, işletim sistemi tüm kesmeleri (interrupts) maskelemeli.

Süreçlerin uygulanması (Implementation of processes):

Süreç Yönetimi (Process management)	Bellek Yönetimi (memory management)	Dosya Yönetimi (File management)
• Kaydediciler (Registers)	• metin Segmenti işaretçisi	• Kök dizin (Root directory)
• Program Sayacı	• Veri Segmenti işaretçisi	• Dosya tanımlayıcılar (File descriptors)
• Program durum sözcüğü	• Yığın (Stack) segmenti "	• Çalışma dizini (Working directory)
• Yığın işaretçisi (SP)		• Kullanıcı kimliği
• Süreç durumu (Process State)		• Grup kimliği
• öncelikli (Priority)		

Bellek: Kaydetmek ya da Kaydetmemek:

- Tüm belleği diske kaydet; Çok zaman alıcı olabilir.
- Belleği kaydetme, bir sonraki sürece güven; PC'ler ve MAC'ler tarafından benimsenen yaklaşım budur.
- Belleği bir sonraki süreçten izole et; Bu bellek yönetimidir, daha sonra ele alınacaktır.

Bağlam değişimi uygulaması (Context Switch implementation): Bağlam değişim mekanizması bir işletim sisteminin en önemli parçasıdır ve uygulanması sırasında özel dikkat gerekir. Çünkü:

- Zorludur (It is tricky): Bir kullanıcı sürecinin durumunu kaydetmek problemlidir. Çünkü işletim sistemi, sürecin mevcut durumunu değiştirmeden durumu kaydetmek için kod çalıştırmalıdır.

- Makineye bağımlıdır (Machine dependent): Teknoloji sayesinde; her CPU, uygulamayı kolaylaştırmak için bazı özel destekler sağlar

Creating a new Process (Yeni bir süreç oluşturma):

• Sıfırdan bir tane oluşturmak (Build one from scratch):
- Kodu ve Veriyi belleğe yükle.
- (Boş) bir dinamik bellek çalışma alanı oluştur.
- Süreç kontrol bloğunu oluştur ve başlat.
- Süreci dağıtıcıya (dispatcher) tanıt.

• Var olan bir tanesini klonlamak (Clone an existing one):
- Mevcut süreci durdur ve durumunu kaydet.
- Kodun, verinin, dinamik bellek çalışma alanının ve süreç kontrol bloğunun bir kopyasını oluştur.
- Süreci dağıtıcıya tanıt.

Process creation mechanisms (Süreç Oluşturma Mekanizmaları):
-> Süreçleri kim oluşturur ve nasıl desteklenirler. Her İşletim sisteminin süreç oluşturmak için bir mekanizması vardır.

Örn: UNIX'te süreç oluşturmak için fork() sistem çağrısı kullanılır. fork(), çağıran sürecin birebir aynı bir kopyasını oluşturur. fork() çağrısından sonra, ebeveyn süreç CPU için eşit şekilde rekabet ettiği çocuğu ile eszamanlı olarak çalışmaya devam eder.

Öte yandan, MS-DOS'ta LOAD_AND_EXEC sistem çağrısı bir çocuk süreç oluşturur. Bu çağrı, çocuk yürütmeyi bitirene kadar ebeveyni askıya alır, bu nedenle ebeveyn ve çocuk eszamanlı olarak çalışmaz

Bir UNIX süreç bağlamı (A UNIX process context): (iki kısım var)

Çekirdekde (in kernel): İşletim sisteminin kendi korumalı alanıdır. Burada daha önce konuştuğumuz süreç kontrol bloğu (PCB) tutulur.

Bağlam Değişimi Yapılabilir Kısım (Context Switchable): Burası kullanıcı alanıdır ve süreç bağlamını (process context) oluşturur. CPU başka bir sürece geçerken bu yapıdaki bilgiler kaydedilir ve yüklenir.

* Kullanıcı alanı kendi içinde şu bölümlerden oluşur:

STACK (Yığın): Yerel değişkenler. Fonksiyon çağrıları ve fonksiyonların içindeki yerel değişkenler burada tutulur.

HEAP (Dinamik Bellek): dinamik değişkenler. Çalışma zamanında kodla dinamik olarak tahsis ettikin bellek alanlarıdır.

DATA (Veri): Sabitler ve statik değişkenler. Global değişkenler ve Program boyunca sabit kalan değerler burdadır.

TEXT (Metin/Kod): Çalıştırılabilir kod. Yazdığın Programın derlenmiş, makine diline çevrilmiş komutları burdadır. Sadece okunabilirdir.

Süreç Sonlandırma (Process Termination):

-> Bir süreç aşağıdaki nedenlerden biri yüzünden çıkış yapıyor durumuna girer:

- Normal Tamamlama: Süreç, sonlanmak için bir sistem çağrısı yürütür.
- Anormal Sonlandırma:
  - Programlama hataları
    - Çalışma zamanı
    - G/Ç (I/O)
  - Kullanıcı müdahalesi

Threads (İş Parçacıkları): Yürütme birimi ve yürütme biriminin ilişkilendirildiği bir kaynaklar koleksiyonu, bir süreç kavramını karakterize eder.

-> Bir iş parçacığı, bir yürütme biriminin soyutlamasıdır. Buna aynı zamanda hafif ağırlıklı süreç (light-weight process) de denir.

-> CPU kullanımının temel birimi olarak bir iş parçacığı; bir talimat işaretçisi, CPU kaydedici seti (register set) ve bir yığından oluşur. Bir iş parçacığı; kodunu ve verilerini, ayrıca sistem kaynaklarını ve diğer işletim sistemiyle ilgili bilgileri eş zamanlıyla paylaşır.

Örn: Bir controller threads, dosya hizmeti isteklerini kabul eder ve her istek için bir worked thread oluşturur. Böylece birçok isteği eş zamanlı olarak işleyebilir. Bir worked thread, bir isteğe hizmet vermeyi bitirdiğinde yok edilir.

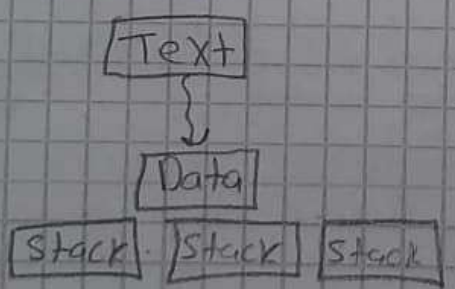
Threads versus Process (İş Parçacıkları ve Süreçlerin Karşılaştırması):

- Bir iş parçacığı büyük ölçüde bir süreçle aynı şekilde çalışır:
  - Çeşitli durumlardan birinde bulunabilir.
  - Sıralı olarak yürütülür.
  - Sistem çağrıları yapabilir.
- Bir iş parçacığı oluşturmak daha az maliyetlidir.
- Bir süreç içindeki bir iş parçacığına geçiş yapmak, farklı süreçlerin iş parçacıkları arasında geçiş yapmaktan daha az maliyetlidir.
- Bir süreç içindeki iş parçacıkları, diğer süreçlerle kıyasla kaynakları kullanışlı ve verimli bir şekilde paylaşırlar.
- Bir süreç içindeki iş parçacıkları bağımsız değillerdir ve birbirlerine karşı korunmazlar.

İş Parçacıkları ve Süreçler:

Geleneksel bir süreç

Çoklu iş parçacıklı bir süreç



İş Parçacığı Modeli (2) (The Thread Model):

Süreç başına düşen öğeler
- Adres alanı
- Global değişkenler
- Açık dosyalar
- Çocuk süreçler
- Bekleyen alarmlar

İş Parçacığı başına düşen öğeler
- Program sayacı
- Kaydedicilik
- Yığın
- Durum

İş Parçacığı uygulamaları (Thread implementations):

• **Kullanıcı Düzeyi (User Level):**
-> Bir dizi kütüphane fonk. olarak uygulanır bağımsız olarak zamanlanmaz; her iş parçacığı bir sürecin zaman dilimini bir kısmını alır; bir iş parçacığı tarafından yapılan bir sistem çağrısı, bir sürecin tüm iş parçacıklarını bloke edeci; daha az maliyetli işlemlerdir.

• **Çekirdek Düzeyi (Kernel Level):**
-> Sistem çağrıları olarak uygulanır; doğrudan işletim sistemi tarafından zamanlanabilir; tek bir süreçteki iş parçacıklarının bağımsız çalışmasını sağlar; daha maliyetli işlemlerdir.

• **Hibrit (Karma) Yaklaşım:**
-> Yukarıdaki iki sistemin avantajlarını birleştirir.